可能更快找到解，但不保证最优

quickly but, unlike the case of an optimistic cost-to-go heuristic, the solution will not be guaranteed to be optimal. One possibility is to run A^* with an inflated cost-to-go to find an initial solution, then rerun the search with progressively smaller values of η until the time allotted for the search has expired or a solution is found with $\eta = 1$.

完备

10.3 Complete Path Planners

数学上很美妙，工程不涉及

Complete path planners rely on an exact representation of the free C-space $\mathcal{C}_{\text{free}}$. These techniques tend to be mathematically and algorithmically sophisticated, and impractical for many real systems, so we do not delve into them in detail.

One approach to complete path planning, which we will see in modified form in Section 10.5, is based on representing the complex high-dimensional space $\mathcal{C}_{\text{free}}$ by a one-dimensional **roadmap** R with the following properties:

(C-space) (R)

① 上高速：$q_{start} \to q'_{start}$

② 走高速：$q'_{start} \to q'_{goal}$ (R)

③ 下高速：$q'_{goal} \to q_{goal}$ (R) (C-space)

可达性 (高速) 路线图 (一维) 线、点 (图、树)

(a) **Reachability**. From every point $q \in \mathcal{C}_{\text{free}}$, a free path to a point $q' \in R$ can be found trivially (e.g., a straight-line path). 从C-space中任何一个q，都能轻易走上R (直线连接)

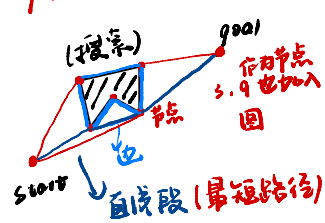
连通性

(b) **Connectivity**. For each connected component of $\mathcal{C}_{\text{free}}$, there is one connected component of R.

With such a roadmap, the planner can find a path between any two points q_{start} and q_{goal} in the same connected component of $\mathcal{C}_{\text{free}}$ by simply finding paths from q_{start} to a point $q'_{\text{start}} \in R$, from a point $q'_{\text{goal}} \in R$ to q_{goal}, and from q'_{start} to q'_{goal} on the roadmap R. If a path can be found trivially between q_{start} and q_{goal}, the roadmap may not even be used.

边：2个节点不与障碍物相交

节点位于C-obs多边形顶点



可视图法

While constructing a roadmap of $\mathcal{C}_{\text{free}}$ is complex in general, some problems admit simple roadmaps. For example, consider a polygonal robot translating among polygonal obstacles in the plane. As can be seen in Figure 10.5, the C-obstacles in this case are also polygons. A suitable roadmap is the weighted undirected **visibility graph**, with nodes at the vertices of the C-obstacles and edges between the nodes that can "see" each other (i.e., the line segment between the vertices does not intersect an obstacle). The weight associated with each edge is the Euclidean distance between the nodes.

可视图：只在obs顶点放节点

Not only is this a suitable roadmap R, but it allows us to use an A^* search to find a shortest path between any two configurations in the same connected component of $\mathcal{C}_{\text{free}}$, as the shortest path is guaranteed either to be a straight line from q_{start} to q_{goal} or to consist of a straight line from q_{start} to a node $q'_{\text{start}} \in R$, a straight line from a node $q'_{\text{goal}} \in R$ to q_{goal}, and a path along the straight edges of R from q'_{start} to q'_{goal} (Figure 10.9). Note that the shortest

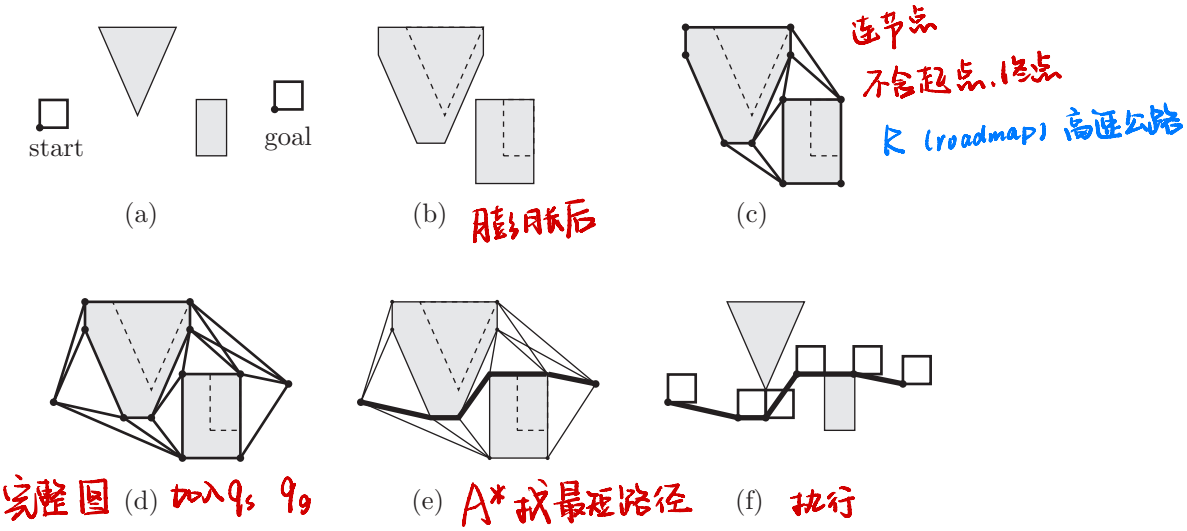


Figure 10.9: (a) The start and goal configurations for a square mobile robot (reference point shown) in an environment with a triangular and a rectangular obstacle. (b) The grown C-obstacles. (c) The visibility graph roadmap R of $\mathcal{C}_{\text{free}}$. (d) The full graph consists of R plus nodes at q_{start} and q_{goal}, along with the links connecting these nodes to the visible nodes of R. (e) Searching the graph results in the shortest path, shown in bold. (f) The robot is shown traversing the path.

临时把边界($\mathcal{C}_{obs}$)也看作安全的

path requires the robot to graze the obstacles, so we implicitly treat $\mathcal{C}_{\text{free}}$ as including its boundary.

10.4 Grid Methods

A search algorithm like A^* requires a discretization of the search space. The simplest discretization of C-space is a grid. For example, if the configuration space is n-dimensional and we desire k grid points along each dimension, the C-space is represented by k^n grid points.

The A^* algorithm can be used as a path planner for a C-space grid, with the following minor modifications:

- The definition of a "neighbor" of a grid point must be chosen: is the robot constrained to move in axis-aligned directions in configuration space or can it move in multiple dimensions simultaneously? For example, for a two-dimensional C-space, neighbors could be 4-connected (on the cardinal points of a compass: north, south, east, and west) or 8-connected